Большакова Маргарита Вадимовна
Гр. 410902

Лабораторная работа № 2
P2P ЧАТ

Задание:

Необходимо разработать программу (консольную или графическую) для обмена текстовыми сообщениями, работающую в локальной сети в одноранговом режиме.

Каждый участник обмена сообщениями (узел) идентифицируется IP-адресом и произвольным именем, которое задается пользователем (через параметр командной строки, конфигурационный файл или любым другим способом). Уникальность имен не требуется.

Каждый узел с помощью UDP формирует список активных узлов (IP-адреса и имена):

после запуска узел отправляет широковещательный пакет, содержащий свое имя, для уведомления других узлов в сети о своем подключении к сети;

другие узлы, получившие такой пакет, устанавливают с отправителем TCP-соединение для обмена сообщениями и передают по нему свое имя для идентификации в чате.

В любой момент к чату может присоединиться новый клиент.

Обмен сообщениями ведется с помощью TCP в логически общем пространстве: каждый узел поддерживает по одному TCP соединению с каждым другим узлом и отправляет свои сообщения всем узлам в сети. Отключение узла должно корректно обрабатываться другими узлами.

Пользовательский интерфейс программы должен позволять вводить с клавиатуры и отправлять сообщения, а также просматривать историю событий с момента последнего запуска программы.

История должна включать следующие сообщения в хронологическом порядке с отметками времени:

- входящие сообщения от других узлов (с указанием имени и IP-адреса отправителя);
- собственные отправленные сообщения;
- обнаружение нового узла;
- отключение работающего узла.

Для обмена сообщениями рекомендуется разработать свой собственный формат сообщения, позволяющий передавать сообщения разных типов и упрощающих передачу сообщений в потоковом режиме, используемом в TCP.

Например:

смещение в байтах	тип сообщения
0	тип сообщения (1 - сообщение, 2 - передача имени, 3 - пользователь подключен, 4 - пользователь отключен т.п.)
1	длина сообщения в байтах далее n байт содержание сообщения

Способы тестирования чата в многопользовательском режиме:

Docker (для консольных программ).

Виртуальные машины VirtualBox (для графических программ).

Bind на разные адреса loopback интерфейса: 127.0.0.1, 127.0.0.2 и т. д.

Поддерживается в новых версиях Windows без дополнительных настроек, может потребовать дополнительных действий в Linux.

При создании сокетов на прослушивание входящих широковещательных UDP-пакетов и TCP-соединений нужно указывать IP-адрес, переданный при запуске программы (параметр командной строки, конфигурационный файл и т.п.). Далее запустить несколько экземпляров программы, передавая разные IP-адреса: 127.0.0.1, 127.0.0.2 и т.д. Все эти адреса будут ассоциированы с вашим компьютером, однако логически программы будут разными независимыми клиентами.

Ход работы:

1) Инициализация узлов и UDP-обнаружение (Handshake)

В начале программы запрашиваются идентификационные данные пользователя (Имя, IP-адрес, UDP и TCP порты). Для отправки и получения широковещательных/направленных запросов знакомства (Handshake) создается UDP-сокет.

```
○ (base) margo@MacBook-Pro-Margarita ЛК3 % /usr/local/bin/python3 "/Users/margo/University 2nd Grad/КСИС/ЛК3/p2p.py"
```

Рисунок 1 – запуск моей программы через терминал Visual Studio Code

```
Введите ваше имя: репа
Введите ваш IP: 127.0.0.1
Введите ваш UDP порт: 5001
Введите ваш TCP порт: 6001

Введите IP друга: 127.0.0.2
Введите UDP порт друга: 5002
```

```
Введите ваше имя: кракен
Введите ваш IP: 127.0.0.2
Введите ваш UDP порт: 5002
Введите ваш TCP порт: 6001

Введите IP друга: 127.0.0.1
Введите UDP порт друга: 5001
```

Рисунок 2 – ввод моих данных через клавиатуру(в двух терминалах)

```
~~~~~~~~~ Привет репа. Твой IP-адрес: 127.0.0.1 ~~~~~~~~~
UDP пакет отправлен
Введите ваше сообщение:
_[+] Найден новый узел кракен : 6001
```

```
~~~~~~~~~ Привет кракен. Твой IP-адрес: 127.0.0.2 ~~~~~~~~~
UDP пакет отправлен
Введите ваше сообщение:
_[+] Найден новый узел репа : 6001
```

Рисунок 3 – вывод на экране после заполнения соответствующих строк

При запуске первый узел сформировал и отправил приветственный UDP-пакет заданному соседу для обмена информацией о портах TCP-сервера.

В программе формирование пакета (включающего служебный байт 03, длину сообщения и полезную нагрузку в виде Имя:Порт) реализовано следующим образом:

```
my_info = f"{my_name}:{my_tcp_port}"
info_encode = my_info.encode('utf-8')
message = bytes([3, len(info_encode)]) + info_encode
udp_sock.sendto(message, (neighbr_ip, neighbr_udp_port))
print("UDP пакет отправлен")
```

1	0.000000	127.0.0.1	127.0.0.2	UDP	47	5001 → 5002 Len=15
2	0.000055	127.0.0.2	127.0.0.1	ICMP	60	Destination unreachable (Port unreachable)
3	37.608032	127.0.0.2	127.0.0.1	UDP	51	5002 → 5001 Len=19
4	37.610409	127.0.0.1	127.0.0.2	UDP	47	5001 → 5002 Len=15
5	37.610482	127.0.0.2	127.0.0.1	UDP	51	5002 → 5001 Len=19

```
> Frame 3: Packet, 51 bytes on wire (408 bits), 51 bytes captured (408 bits) on interface lo0,
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.2, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 5002, Dst Port: 5001
> Data (19 bytes)
```

2) Инициализация обмена сообщениями и тройное TCP-рукопожатие

После успешного обнаружения узлов была произведена попытка отправки текстового сообщения от одного пользователя к другому. На транспортном уровне это инициировало установку надежного соединения.

```
[TCP] установлено соединение с ('127.0.0.2', 57703)

 [Неизвестный - 127.0.0.2] : Приветики

Соединение с ('127.0.0.2', 57703) разорвано
```

При тестировании двух узлов на одном компьютере (на macOS). Когда мы тестируем сеть внутри одной машины, операционная система оптимизирует маршруты. Даже если я указывала узлу IP-адрес 127.0.0.2 (или адрес Wi-Fi), при установке реального TCP-соединения ядро системы могло направить трафик через стандартный адрес петли (Loopback) — 127.0.0.1. Из-за этого IP-адрес отправителя в TCP-пакете не совпадал с тем, что был сохранен в словаре на этапе UDP-разведки. Обработка неизвестных ключей спасла программу от сбоев в этой ситуации. Поэтому Я реализовала вывод "Неизвестный" через метод .get() чтобы не возникало ошибки(при тестировании выдавало ошибку KeyError)

```
[TCP] установлено соединение с ('127.0.0.1', 57704)

 [кракен - 127.0.0.1] : Приветики

Соединение с ('127.0.0.1', 57704) разорвано
```

Для передачи информации о TCP-сервере узла был разработан собственный формат пакета. Он состоит из служебных байтов (тип сообщения 3, длина полезной нагрузки) и самих данных в формате Имя:TCP-порт.

```
send_sock.connect((find_ip, neighbrs[find_ip][1])) #подключились к соседу. Клиентская часть (отправка)
```

```
# Серверная часть (прием)
client_sock, client_addr = tcp_server.accept()
```

45	481.203795	127.0.0.1	127.0.0.1	TCP	68	57704 → 6001 [SYN] Seq=0 Win=6
46	481.203835	127.0.0.1	127.0.0.1	TCP	68	6001 → 57704 [SYN, ACK] Seq=0
47	481.203843	127.0.0.1	127.0.0.1	TCP	56	57704 → 6001 [ACK] Seq=1 Ack=1

```
74  57704 → 6001  [PSH, ACK]
56  57704 → 6001  [FIN, ACK]
```

На рисунке представлен дамп TCP-трафика, полученный при отправке одного сообщения. Пакеты демонстрируют успешную установку соединения (тройное рукопожатие). Пакет с флагом PSH содержит полезную нагрузку (текст сообщения). Пакеты иллюстрируют процесс двустороннего разрыва соединения с использованием флагов FIN и ACK, инициированный функцией shutdown().

Во избежание потери данных использовался метод shutdown, инициирующий отправку FIN-пакета, после чего сокет полностью закрывался вызовом close():

```
send_sock.shutdown(socket.SHUT_RDWR)
send_sock.close()
```

Вывод: в ходе выполнения лабораторной работы было успешно разработано многопоточное P2P-приложение, реализующее гибридную клиент-серверную архитектуру. Разработан собственный прикладной протокол: для автоматического обнаружения узлов применяется широковещательная рассылка через UDP-сокеты, а для гарантированной доставки текстовых сообщений устанавливается надежное соединение через TCP. Практическая реализация проекта позволила на глубоком уровне закрепить принципы многопоточного программирования, управления сетевыми гнездами (сокетами) и изучить особенности маршрутизации в локальных сетях с учетом масок подсетей и транспортных протоколов.